

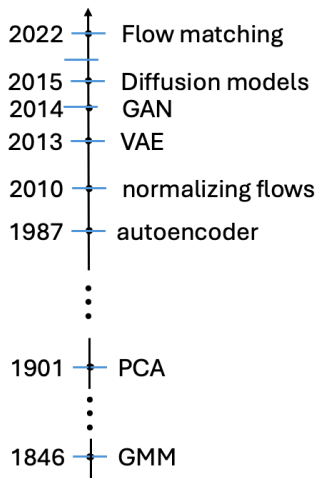
COMM114: Generative AI - Normalizing Flows (part 2)

Guoqiang Zhang

November 2025

Recap from last lecture

Normalizing flow (NF) was invented in 2010.



Recap from last lecture

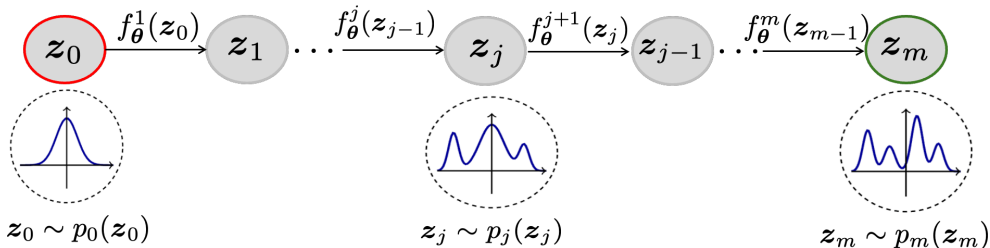
In normalizing flow (NF), an explicit probability density function (PDF) can be obtained after applying an invertible mapping.

- Input: data samples $\{\mathbf{x}_i\}_{i=1}^n$
- Let $\mathbf{z} \sim \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$.
- Let $f_{\boldsymbol{\theta}} : \mathbb{R}^d \rightarrow \mathbb{R}^d$ be differentiable and invertible. Its inverse function is $h_{\boldsymbol{\theta}}(\cdot) = f_{\boldsymbol{\theta}}^{-1}(\cdot)$.
- The parameter $\boldsymbol{\theta}$ in $f_{\boldsymbol{\theta}}$ can be learned by maximizing the log-likelihood via

$$\begin{aligned} & \arg \max_{\boldsymbol{\theta}} \frac{1}{n} \sum_{i=1}^n \log(p_{\mathbf{X}}^{\boldsymbol{\theta}}(\mathbf{x}_i)) \\ &= \arg \max_{\boldsymbol{\theta}} \frac{1}{n} \sum_{i=1}^n \left(\log p_{\mathbf{Z}}(h_{\boldsymbol{\theta}}(\mathbf{x}_i)) + \log |\det D h_{\boldsymbol{\theta}}(\mathbf{x}_i)| \right) \\ &= \arg \max_{\boldsymbol{\theta}} \frac{1}{n} \sum_{i=1}^n \left(\log p_{\mathbf{Z}}(h_{\boldsymbol{\theta}}(\mathbf{x}_i)) - \log |\det D f_{\boldsymbol{\theta}}(\mathbf{x}_i)| \right) \end{aligned}$$

Recap from last lecture

- Input: data samples $\{x_i\}_{i=1}^n$
- Let $z \sim \mathcal{N}(z; \mathbf{0}, I)$.
- Let $f_{\theta}(z) = f_{\theta}^m \circ \dots \circ f_{\theta}^1(z)$
 - Denote the inverse function of each individual function $f_{\theta}^j(z)$ as $h_{\theta}^j(z)$



Recap from last lecture

- Input: data samples $\{\mathbf{x}_i\}_{i=1}^n$
- Let $\mathbf{z} \sim \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$.
- Let $f_{\boldsymbol{\theta}}(\mathbf{z}) = f_{\boldsymbol{\theta}}^m \circ \dots \circ f_{\boldsymbol{\theta}}^1(\mathbf{z})$
 - Denote the inverse function of each individual function $f_{\boldsymbol{\theta}}^j(\mathbf{z})$ as $h_{\boldsymbol{\theta}}^j(\mathbf{z})$
- The parameter $\boldsymbol{\theta}$ in $f_{\boldsymbol{\theta}}$ can be learned by maximizing the log-likelihood via

$$\begin{aligned} & \arg \max_{\boldsymbol{\theta}} \frac{1}{n} \sum_{i=1}^n \log(p_{\mathbf{X}}^{\boldsymbol{\theta}}(\mathbf{x}_i)) \\ &= \arg \max_{\boldsymbol{\theta}} \frac{1}{n} \sum_{i=1}^n \left(\log p_{\mathbf{Z}}(h_{\boldsymbol{\theta}}(\mathbf{x}_i)) + \sum_{j=1}^m \log |\det D h_{\boldsymbol{\theta}}^j(\mathbf{z}_{ij})| \right) \quad [\mathbf{z}_{im} = \mathbf{x}_i] \\ &= \arg \max_{\boldsymbol{\theta}} \frac{1}{n} \sum_{i=1}^n \left(\log p_{\mathbf{Z}}(h_{\boldsymbol{\theta}}(\mathbf{x}_i)) - \sum_{j=1}^m \log |\det D f_{\boldsymbol{\theta}}^j(\mathbf{z}_{ij})| \right) \end{aligned}$$

Objective of this lecture

- To study different reversible neural architectures when constructing normalizing flows.
- To study how the Jacobian matrix is computed for each reversible neural architecture.

Linear Flows

- A linear transformation can be a flow if the matrix is invertible

$$\boldsymbol{x} = f(\boldsymbol{z}) = A\boldsymbol{z} + b$$

- Its inverse $h(\boldsymbol{x}) = f^{-1}(\boldsymbol{x})$ is

$$\boldsymbol{z} = h(\boldsymbol{x}) = f^{-1}(\boldsymbol{x}) = A^{-1}(\boldsymbol{x} - b)$$

- Determinant of the Jacobian matrix for $f(\boldsymbol{z})$:

$$\det Df(\boldsymbol{z}) = \det A$$

- Determinant of the Jacobian matrix for $h(\boldsymbol{x})$:

$$\det Dh(\boldsymbol{x}) = \det(A^{-1})$$

Linear Flows

- Problem: Inexpressive
 - Composition of a set of linear functions is still a linear function.
 - Linear flow alone cannot build advanced normalizing flow models.
 - Complexity for computing the inverse A^{-1} and the determinant is $O(d^3)$, where d is either the number of rows or the number of columns in A .
- Restricting the form of A matrix to reducing complexity

	inverse	Determinant
Full	$O(d^3)$	$O(d^3)$
Diagonal	$O(d)$	$O(d)$
Triangular	$O(d^2)$	$O(d)$
Block Diagonal	$O(c^3d)$	$O(c^3d)$

Remark: c is the size of the each submatrix in a block diagonal matrix.

Coupling Flows

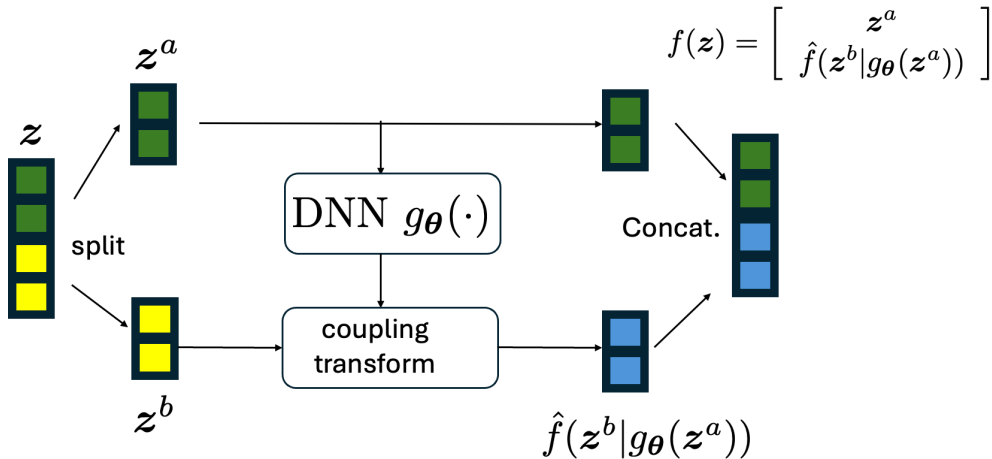
- A coupling flow is a general approach to construct non-linear flows
- Partition the vector z into two disjoint subvectors $z = (z^a, z^b)$ then

$$f(z) = \begin{pmatrix} z^a \\ \hat{f}(z^b | g_\theta(z^a)) \end{pmatrix} \in \begin{pmatrix} \mathbb{R}^{d/2} \\ \mathbb{R}^{d/2} \end{pmatrix}$$

- g_θ is a neural network.
- $\hat{f}$ function is reversible. That is, z^b can be recovered with the output of the function and $g_\theta(z^a)$.

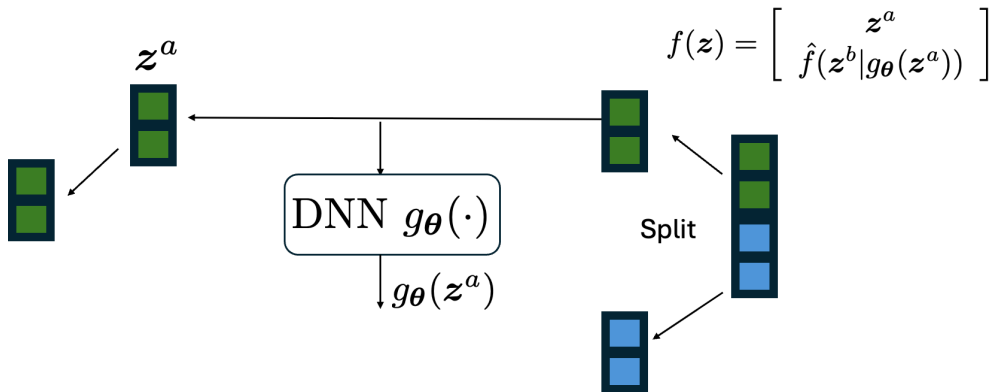
Coupling Flows (cont.)

- Demonstration of a coupling flow:



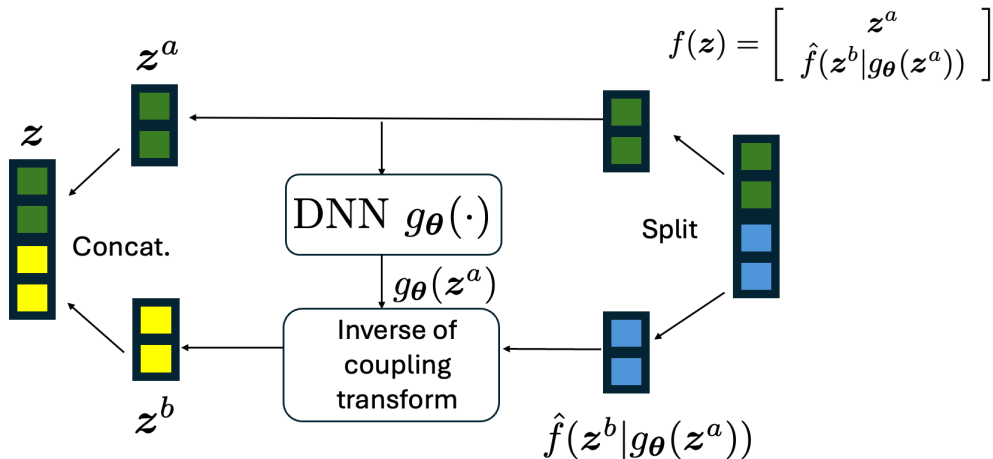
Coupling Flows: reverse

- Objective: Compute the input z given the output $f(z) = \begin{pmatrix} z^a \\ \hat{f}(z^b | g_\theta(z^a)) \end{pmatrix}$.
- Step 1: Recover z^a



Coupling Flows: reverse

- Step 2: Recover z^b



Remark: We will talk about the particular forms of the coupling transform later on.

Coupling Flows: Jacobian

- Suppose the update expression of a coupling flow is given below:

$$f(\mathbf{z}) = \begin{pmatrix} \mathbf{z}^a \\ \hat{f}(\mathbf{z}^b | g_{\boldsymbol{\theta}}(\mathbf{z}^a)) \end{pmatrix}$$

- Its Jacobian matrix is block-diagonal.

$$\begin{aligned} Df(\mathbf{z}) &= \begin{bmatrix} \frac{\partial \mathbf{z}^a}{\partial \mathbf{z}^a} & \frac{\partial \mathbf{z}^a}{\partial \mathbf{z}^b} \\ \frac{\partial \hat{f}(\mathbf{z}^b | g_{\boldsymbol{\theta}}(\mathbf{z}^a))}{\partial \mathbf{z}^a} & \frac{\partial \hat{f}(\mathbf{z}^b | g_{\boldsymbol{\theta}}(\mathbf{z}^a))}{\partial \mathbf{z}^b} \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \frac{\partial \hat{f}(\mathbf{z}^b | g_{\boldsymbol{\theta}}(\mathbf{z}^a))}{\partial \mathbf{z}^a} & \frac{\partial \hat{f}(\mathbf{z}^b | g_{\boldsymbol{\theta}}(\mathbf{z}^a))}{\partial \mathbf{z}^b} \end{bmatrix} \end{aligned}$$

Remark: In practice, we are interested in the determinant of the Jacobian matrix when training a normalizing flow model.

Coupling Flows: Jacobian

- Given the Jacobian matrix:

$$\begin{aligned} Df(\mathbf{z}) &= \begin{bmatrix} \frac{\partial \mathbf{z}^a}{\partial \mathbf{z}^a} & \frac{\partial \mathbf{z}^a}{\partial \mathbf{z}^b} \\ \frac{\partial \hat{f}(\mathbf{z}^b | g_{\boldsymbol{\theta}}(\mathbf{z}^a))}{\partial \mathbf{z}^a} & \frac{\partial \hat{f}(\mathbf{z}^b | g_{\boldsymbol{\theta}}(\mathbf{z}^a))}{\partial \mathbf{z}^b} \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \frac{\partial \hat{f}(\mathbf{z}^b | g_{\boldsymbol{\theta}}(\mathbf{z}^a))}{\partial \mathbf{z}^a} & \frac{\partial \hat{f}(\mathbf{z}^b | g_{\boldsymbol{\theta}}(\mathbf{z}^a))}{\partial \mathbf{z}^b} \end{bmatrix} \end{aligned}$$

- The determinant $\det(Df(\mathbf{z}))$ takes the form

$$\det(Df(\mathbf{z})) = \det \left(\frac{\partial \hat{f}(\mathbf{z}^b | g_{\boldsymbol{\theta}}(\mathbf{z}^a))}{\partial \mathbf{z}^b} \right)$$

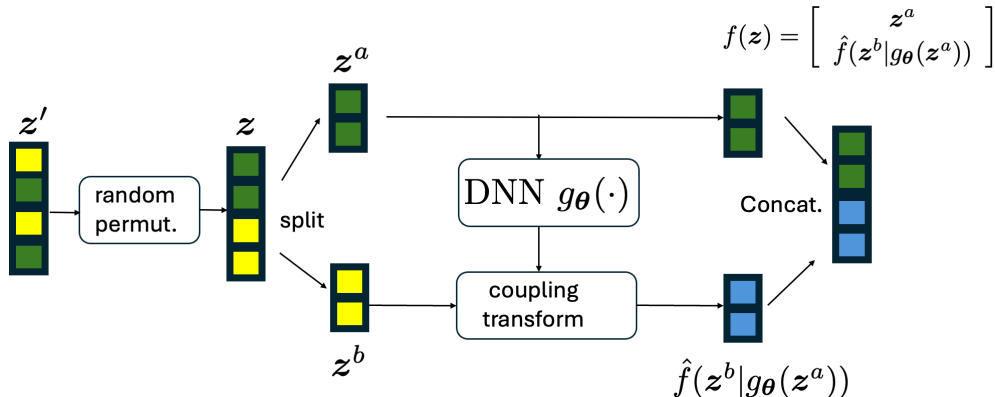
- The determinant $\det(Df(\mathbf{z}))$ is independent of $\frac{\partial \hat{f}(\mathbf{z}^b | g_{\boldsymbol{\theta}}(\mathbf{z}^a))}{\partial \mathbf{z}^a}$.

A note on g_θ

- The determinant $\det(Df(z))$ is not directly related to the DNN model g_θ .
- In practice, we can make $g_\theta(x^a)$ arbitrarily complex, e.g., MLP, CNN, Transformer, etc.
- The purpose for introducing a complex DNN model is to increase the expressiveness of the overall normalizing flow (NF) model. That is, to make the NF model to approximate more complex data distribution p_{data} if needed.

Permutation is needed in Coupling Flows

- Before a passing a tensor to a coupling flow layer, the elements in the tensor needs to be permuted.
- Without permutation, the first half z^a would keep the same across all coupling flow layers in a NF model.



Coupling Flows - Coupling Transforms

- Additive[NICE, Dinh et al 2014]

$$\hat{f}(z^b | g_{\theta}(z^a)) = z^b + g_{\theta}(z^a)$$

- Recover z^b by using $g_{\theta}(z^a)$:

$$z^b = \hat{f}(z^b | g_{\theta}(z^a)) - g_{\theta}(z^a)$$

Coupling Flows - Coupling Transforms (cont.)

- Affine [RealNVP, Dinh et al 2016]

$$\hat{f}(z^b | g_{\theta}(z^a), s(z^a)) = \exp(s(z^a)) \odot z^b + g_{\theta}(z^a)$$

- $s(z^a) \in \mathbb{R}^{d/2}$ is a vector of the same dimensionality as z^a .
- $s(z^a)$ can be obtained as another DNN model with z^a as input to the DNN model.
- $\odot$ denotes element-wise multiplication.
- Recover z^b by using $g_{\theta}(z^a)$:

$$z^b = \hat{f}(z^b | g_{\theta}(z^a)) / \exp(s(z^a)) - g_{\theta}(z^a) / \exp(s(z^a))$$

•

•

Coupling Flows - Coupling Transforms (cont.)

- Affine [RealNVP, Dinh et al 2016]

$$\hat{f}(z^b | g_{\theta}(z^a), s(z^a)) = \exp(s(z^a)) \odot z^b + g_{\theta}(z^a)$$

- Determinant of the Jacobian

$$\det \left(\frac{\partial \hat{f}(z^b | g_{\theta}(z^a), s(z^a))}{\partial z^b} \right) = \prod_{j=1}^{d/2} \exp(s_j(z^a)) = \exp \left(\sum_{j=1}^{d/2} s_j(z^a) \right)$$

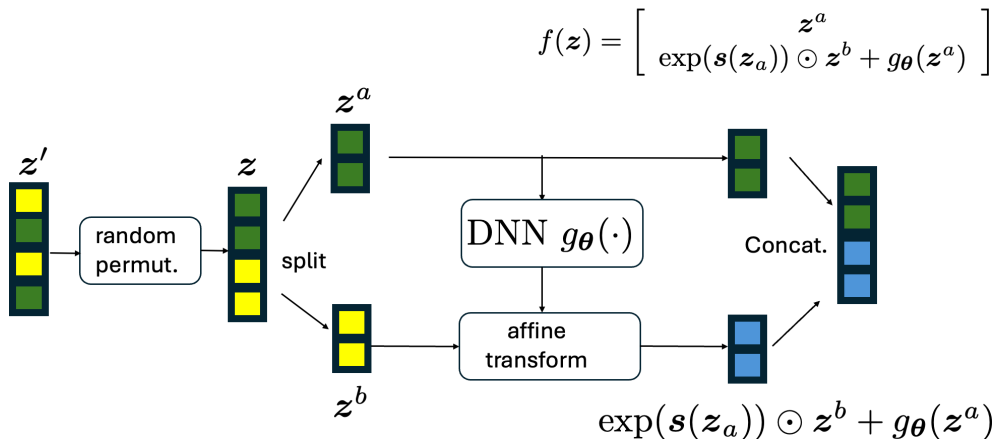
- s_j is the j th element of $s(z^a)$.
- Log Det of the inverse Jacobian is the negative of the Log-Det Jacobian:

$$\log \frac{1}{\exp(\sum_{j=1}^{d/2} s_j(s(z^a)))} = - \left(\sum_{j=1}^{d/2} s_j(z^a) \right)$$

Remark: [website-link](#)

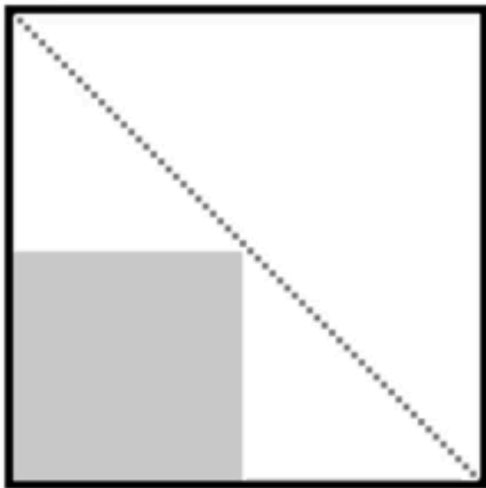
Coupling Flows

- Coupling flow with an affine transformation:



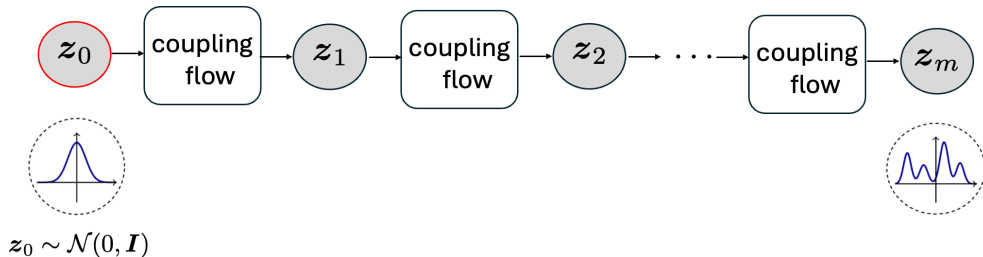
Coupling Flows

- Coupling flow with an affine transformation has a very sparse Jacobian matrix:
- Only a sub-block in the Jacobian matrix is dense. This suggests that the expressiveness of the coupling flow is limited.



A NF model consisting of m coupling flows

- By concatenating a number of coupling-flow layers sequentially, one obtain the final NF model.



Remark: Check last lecture on how to train the NF model.

References

- Lecture notes by Stefano Ermon
- Youtube talk by Marcus Brubaker

Questions?